

Trajector: Pathwise Adjoint Correction of Step-Cached Diffusion Transformers

Ruize Xia

High School Student

Nanjing Foreign Language School

Nanjing, China

xiaruize0911@gmail.com

Abstract—Step caching accelerates diffusion transformers by reusing a block residual across neighbouring solver steps. The speedup needs no training, but the image degrades. A learned corrector can predict that reuse error; the standard way to train it is residual regression. We show that this objective is misaligned with deployed image quality: cache error integrates along the sampling trajectory, because an anchor refreshes the cache but not the latent, so the residual target at each step is defined on a trajectory the deployed corrector never visits. Driving residual accuracy 25–43% higher by retraining on matched data buys only +0.005 to +0.013 SSIM. We therefore fine-tune the same corrector with Trajector (*pathwise adjoint correction*), which minimises the deviation of the latent trajectory by differentiating through the cached rollout, in about twenty minutes of training. Trajector improves image quality at every injection granularity we tested ($K=1$ to 28, all significant, $t=2.7$ to 7.0, $n=72$). At a fixed step count on PixArt- Σ -XL-2-512, Trajector reaches SSIM 0.6820 against the exact output at $3.41\times$ speedup. It outperforms a reproduction of TaylorSeer’s extrapolation mechanism by +0.0946 SSIM ($t=14.5$, 72 of 72 paired cases) at an 8.4% speedup cost, and a reproduction of Block Caching by +0.0490 ($t=8.4$) while running 12% faster. Under ImageReward both reproduced mechanisms score *below* verbatim caching, and learned correction is the only method that improves on it.

Index Terms—diffusion transformers, feature caching, pathwise objectives, inference acceleration

1. Introduction

Sampling a diffusion transformer [1] costs one full stack forward pass per solver step. Step caching exploits the smoothness of that trajectory: an *anchor* step, taken every i steps, stores each block’s residual $R = \text{block}(h) - h$ for input h , and the next $i - 1$ *reuse* steps skip the block and add the stored residual instead [2], [3], [4]. The method needs no training and is fast, but it visibly degrades the image.

Two families of remedies exist. Training-free extrapolation forecasts residual drift from its own history [5], [6].

Learned correction trains a small network [7], [8] to predict the reuse error

$$\Delta R = R_{\text{true}} - R_{\text{anchor}}, \quad (1)$$

where R_{true} is the residual the block would produce on the current hidden state and R_{anchor} is the one stored at the most recent anchor. ΔR is exactly the error verbatim reuse commits. This paper studies the second family. Its default training signal is supervised regression on (1), and every later design choice (architecture, checkpoint selection, and how much correction to inject) is then made in residual space.

We show that this signal is misaligned with deployed quality, and that a cheap second training stage fixes much of it. Concretely, we contribute:

- 1) **A diagnosis of the residual objective.** The cached trajectory’s deviation from the exact one climbs almost monotonically (0.0089 at step 1 to 0.6092 at step 19): an anchor refreshes the cache, not the latent. Residual error is pointwise; trajectory error integrates. Retraining with matched data to convergence improves residual error by 25–43% but image SSIM by only +0.005 to +0.013.
- 2) **Trajector.** Keeping the architecture, schedule and deployment rule, we fine-tune the corrector by differentiating through the cached rollout. A cached block never runs, so the graph over one reuse step is K corrector forwards rather than K blocks of a 0.6B model; training takes about 20 minutes. It improves image quality at every K , and most where residual training degrades most.
- 3) **A comparison with published mechanisms.** At a fixed step count and skip schedule, Trajector outperforms reproductions of TaylorSeer and Block Caching on SSIM, PSNR and ImageReward.

2. Related Work

2.1. Fewer Steps Versus Cheaper Steps

Latent diffusion [9] and DiT [1] made iterative sampling the default; PixArt- Σ [10] and rectified-flow transformers [11], [12] keep that cost structure. High-order ODE solvers

[13], [14], [15], [16] and distillation [17], [18], [19] shorten the trajectory; caching makes each remaining step cheaper. We hold the step count and solver fixed, which isolates caching but does not rank the two routes (Sec. 5.3).

2.2. Training-Free Feature Caching

DeepCache [2] reuses U-Net skip features. On transformers the unit is a block residual: FORA [3] on a fixed interval; Δ -DiT [4] caches different blocks early and late; TeaCache [5], ToCa [20], FasterCache [21] and AdaCache [22] adapt *when* to skip, then reuse verbatim. Our baseline caches the whole stack at uniform intervals, with no correction. We take the schedule as given and ask what to add back.

2.3. Learned Corrections and Training Objectives

Learning-to-Cache [8] learns which layers may be skipped. LazyDiT [23] places a linear predictor in front of each block and reuses from a first-order Taylor residual. Block Caching [7] learns a per-channel scale and shift for each timestep: trained, but a function of the timestep alone. TaylorSeer [6] replaces verbatim reuse with a forecast of the cached residual from finite differences, with no training. Our corrector differs from all four in reading the *current state*. That is what makes error propagation across injection sites multiplicative (Sec. 5.2), and it is why the training objective matters: a correction that ignores the state cannot be improved by observing the trajectory it produces. HarmoniCa [24] already names an “objective mismatch” between aligning predicted noise and generating good images, and fixes it by training along the denoising process with a proxy for final image error in the objective. We do not claim the mismatch itself as novel. What we add is a cheaper and more local diagnosis (residual error is pointwise, while cache error integrates along the latent trajectory) and a fix that needs no proxy in image space: the cached rollout is already differentiable, so the trajectory deviation can be the loss. P2 [25] and Min-SNR [26] reweight the residual target; they do not replace it. DAGger [27] and scheduled sampling [28] correct a closed-loop data distribution. Here the binding constraint is the objective, and the cached rollout is already cheap to differentiate, so the resampling detour is unnecessary.

3. The Residual Objective

The measurements below use PixArt- Σ -XL-2-512 [10] (0.6B parameters, 28 blocks) with its shipped DPM-Solver++ scheduler [15] at 20 steps and 512^2 . Anchors are uniform at interval $i=5$ (4 anchor steps, 16 reuse steps). All numbers are against the exact 20-step output from the same seed. The 28 blocks are grouped into K contiguous cached segments, so K is the number of injection sites per reuse step: $K=1$ corrects the whole stack once and $K=28$ corrects every block.

3.1. Trajectory Error

At each of the 20 steps we measure $\|x_{\text{cached}} - x_{\text{exact}}\|/\|x_{\text{exact}}\|$. Deviation rises from 0.0089 to 0.6092, with only small dips at the four anchors. This is the basic reason the residual target is misaligned: the corrector’s action at step t changes the state at $t+1$, while (1) is defined on the unperturbed trajectory and (3) never differentiates through (2).

3.2. Residual Retraining

To check that (3) had not simply been trained too little, we matched data volume at 50,176 slots per rung (seed multiplier $28/K$), subsampled 48 of 1024 tokens per site, and doubled the step budget until the gain fell below 0.5%. Residual error improved 25–43%. Image SSIM improved by +0.0131 at $K=1$, to +0.0055 at $K=7$: significant, but small. Residual improvement rises with K while the image quality it buys falls.

3.3. Oracle Correction

An oracle that recovers a fraction β of the true residual has residual error $(1 - \beta)^2$. Trajectory error at the last step falls from 0.6102 at $\beta=0$ to 0.4384 at $\beta=0.8$, and to 0 at $\beta=1$. The curve is convex: the ceiling is perfect and the middle is flat, which rules out the correction *form* as the limit and locates the problem in the objective. Trained correctors additionally beat an oracle at their own equivalent β by about $1.5\times$, because a scalar oracle shrinks error isotropically while a trained corrector learns which directions matter to the trajectory. Residual error therefore distorts both ways: as an objective it overstates the gain, as a capability measure it understates the corrector.

4. Method

4.1. Step Caching

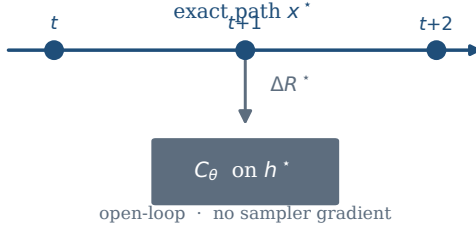
A DiT block b is residual, $R_b(h) = \text{block}_b(h) - h$, where h is its input hidden state (1024 tokens $\times$ 1152 channels), so one exact block maps $h \mapsto h + R_b(h)$. Let $t = 0, \dots, T-1$ index solver steps ($T=20$) and $\mathcal{A}$ be the anchor set (here $\{0, 5, 10, 15\}$ at interval $i=5$). On $t \in \mathcal{A}$ every block runs and the residual is stored. On a reuse step the stored residual is added instead:

$$h \mapsto h + R_b(h_a), \quad a = \max\{s \in \mathcal{A} : s \leq t\}, \quad (2)$$

where a is the latest anchor and h_a the input the block saw there. With blocks grouped into K sites, the reuse error at site k of step t is $\Delta R_{t,k} = R_{t,k}^{\text{true}} - R_{t,k}(h_a)$, i.e. (1). The residual objective regresses the corrector’s prediction $\widehat{\Delta R}_{t,k}$ onto it on the *unperturbed* hidden states of the exact trajectory,

$$\mathcal{L}_{\text{res}} = \sum_{t \notin \mathcal{A}} \sum_{k=1}^K \frac{\|\Delta R_{t,k} - \widehat{\Delta R}_{t,k}\|^2}{\|\Delta R_{t,k}\|^2}. \quad (3)$$

(a) Residual regression



(b) Trajectory

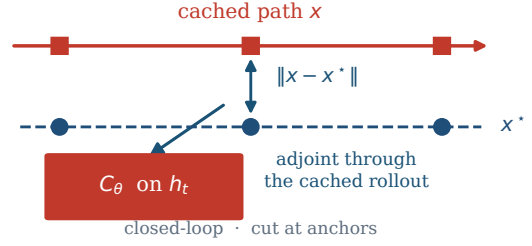


Figure 1. (a) Residual regression supervises C_θ on the exact path. (b) Trajectory trains C_θ by the discrete adjoint of the cached rollout. Architecture and the skip schedule are in the text.

No derivative is taken through (2), so (3) cannot see that a correction at step t changes the state at $t+1$.

4.2. Corrector

The 28 blocks are partitioned into K contiguous cached segments. A shared network C_θ is applied at each site. It reads the state drift $\Delta h = h_t - h_a$, the cached residual R_a and the anchor hidden state h_a , conditioned on the diffusion timestep t , the reuse horizon $\tau = t - a$ and the site index k . Each input is divided by its RMS over the training set ($s_{\Delta h}$, s_R , s_h) and the head is rescaled by the target RMS $s_{\Delta R}$,

$$\widehat{\Delta R} = s_{\Delta R} C_\theta \left(\frac{\Delta h}{s_{\Delta h}}, \frac{R_a}{s_R}, \frac{h_a}{s_h} \middle| t, \tau, k \right), \quad (4)$$

without which the three inputs differ in scale by up to $170\times$. Conditioning is $c = \text{MLP}([\gamma(t); \gamma(\tau); e_k])$ with Fourier features γ and a site embedding e_k . The trunk is four AdaLN residual MLP blocks of width 512, one 4-head attention layer across tokens, and a rank-256 linear path for each site. The output projection is zero-initialised, so $C_\theta=0$ is verbatim reuse (11.9M parameters at $K=1$, 43.7M at $K=28$). Deployment is

$$h_{\text{out}} = h_t + R_a + \sigma \widehat{\Delta R}, \quad (5)$$

with injection strength σ selected on validation, never on test. The Jacobian $J_k = \partial \widehat{\Delta R}_k / \partial h$ is nonzero because C_θ reads Δh ; that is what turns error across sites multiplicative (Sec. 5.2).

4.3. Pathwise Adjoint Training

Trajectory keeps (4)–(5) and replaces (3). Let x_t be the latent after step t of the cached rollout under (5) and x_t^* the exact latent under the same prompt and seed. The pathwise objective is

$$\mathcal{L} = \sum_{t \notin \mathcal{A}} \frac{\|x_t - x_t^*\|^2}{\|x_t^*\|^2}, \quad (6)$$

minimised by reverse-mode differentiation through the cached rollout, which is the discrete adjoint of the sampler restricted to reuse steps (Fig. 1b). Supervision is free: x^* is produced by the same forward pass that yields C_θ 's input.

The adjoint is cheap for a reason specific to caching. A cached block never runs block_b ; the runtime returns (5), so the graph over one reuse step is K applications of C_θ , not K blocks of a 0.6B DiT. Peak memory is 7.5GB at $K=1$; checkpointing each corrector call brings $K=14$ from over 44GB to 4.2GB. Gradients are cut at anchors, where the cache is refreshed from exact values, so a corrector call influences later intervals only through the latent. Extending the graph across intervals improves (6) but not image metrics; we keep the graph to one interval.

Trajectory is a second training stage, not a replacement trained from scratch. Residual regression is a cheap, teacher-forced way to reach a corrector that already helps; we argue it is the wrong *final* objective, not a useless one. We therefore initialise from the residual corrector of the same K retrained on matched data (Sec. 3), already trained until more steps stopped helping; it is also the “Residual obj.” arm below, so each comparison isolates the second stage. We did not test pathwise training from the zero-initialised corrector. Training uses AdamW, learning rate 10^{-5} , gradients clipped to norm 1.0, 400 steps, one prompt per step. C_θ is held in fp32 against an fp16 transformer. Cost is measured: 2.57 TFLOP per anchor step, 0.041 TFLOP per corrector call, 2.51 TFLOP fixed per image. One run is about 20 minutes on an A40.

5. Experiments

Every image metric is a paired comparison over 24 held-out prompts $\times$ 3 seeds ($n=72$), with σ re-selected on validation for each corrector over a common grid. We report SSIM [29], PSNR, LPIPS [30] and ImageReward [31], together with t statistics and the number of cases that improve. Prompts are templated (10 styles $\times$ 24 subjects $\times$ 10 conditions); the held-out split shares no subject with training.

5.1. Comparison at a Fixed Step Count

Table 1 compares at 20 steps and $i=5$, so every arm skips the same blocks and speedup differs only by each method's correction overhead. We reproduce the TaylorSeer mechanism [6] (a forecast of the residual from finite differences of its own history, with no training) and the Block Caching mechanism [7] (a per-channel scale and shift for each timestep,

TABLE 1. FIXED 20 STEPS, $i=5$, $n=72$. “Sp.” IS MEASURED WALL-CLOCK SPEEDUP AGAINST THE EXACT MODEL, IR IS IMAGEREWARD [31], AND Δ SSIM IS PAIRED AGAINST VERBATIM CACHING. THE TWO REPRODUCED MECHANISMS ARE DESCRIBED IN THE TEXT.

Method	Sp.	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	IR $\uparrow$	Δ SSIM
Exact, 20 steps	1.72	0.7951	21.78	0.1786	—	—
Verbatim cache	3.79	0.6269	15.37	0.4669	+0.74	—
TaylorSeer 1st	3.72	0.5874	14.31	0.5080	+0.58	-0.0395
TaylorSeer 2nd	3.73	0.4782	13.57	0.6386	-0.32	-0.1487
Block Caching	3.04	0.6330	15.66	0.4594	+0.59	+0.0060
Residual obj.	3.41	0.6676	16.48	0.3795	+1.07	+0.0407
Trajector	3.41	0.6820	17.20	0.3883	+1.08	+0.0551

TABLE 2. THE TWO OBJECTIVES AT EACH GRANULARITY, $n=72$, EACH AT THE σ SELECTED ON VALIDATION.

K	Residual obj.	Trajector	Difference
1	+0.0406	+0.0553 ($\sigma=0.60$)	+0.0147 ($t=2.7$)
2	+0.0149	+0.0385 ($\sigma=0.30$)	+0.0236 ($t=3.8$)
4	+0.0189	+0.0450 ($\sigma=0.22$)	+0.0261 ($t=5.0$)
7	+0.0145	+0.0515 ($\sigma=0.18$)	+0.0370 ($t=7.0$)
14	+0.0096	+0.0414 ($\sigma=0.18$)	+0.0318 ($t=4.7$)
28	+0.0148	+0.0380 ($\sigma=0.18$)	+0.0232 ($t=4.8$)

trained but blind to the state). Both are reimplemented in our runtime rather than run from the authors’ code, so they are reproductions of the mechanisms under our schedule and backbone, not replications of the published results.

Paired against Trajector, the first-order TaylorSeer mechanism loses 0.0946 SSIM ($t=14.5$) and 0.1197 LPIPS ($t=14.2$), with Trajector better on all 72 of 72 cases; the second-order version loses 0.2038 ($t=39.8$, 72/72); Block Caching loses 0.0490 ($t=8.4$, 64/72), while Trajector is 12% faster than Block Caching. Correction trained on residuals already outperforms both reproduced mechanisms; Trajector widens that lead.

The ImageReward column reverses one fidelity verdict. Both reproduced mechanisms score *below* verbatim caching (-0.159 and -0.155 , $t=-2.1$). Block Caching looks harmless on SSIM (+0.0060) yet is worse than not correcting once a preference model scores the image. Against it Trajector gains +0.492 ($t=6.3$, 60/72), and against first-order TaylorSeer +0.497 ($t=6.0$, 60/72). Only the two learned correctors improve on verbatim caching under ImageReward (+0.325 and +0.338).

Two caveats. TaylorSeer at $i=5$ is worse than no correction (-0.0395 , 3/72); it helps at $i=2-3$ in our reproduction, so Table 1 is not a claim against its own operating point. It also needs no training, while Trajector costs 11.9M parameters and 0.041 TFLOP per correction, which accounts for the 8.4% speedup gap.

5.2. Injection Granularity

Table 2 and Fig. 2 show the objective change helping at all six granularities, every difference significant. The margin

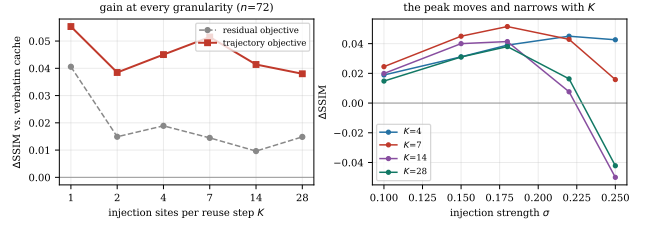


Figure 2. Left: gain over verbatim caching at each granularity under the two objectives, each at the σ selected on validation. Right: sensitivity to σ for the high- K correctors; the optimum moves left and the peak narrows as K grows, and $K=14$ and $K=28$ turn negative by $\sigma=0.25$.

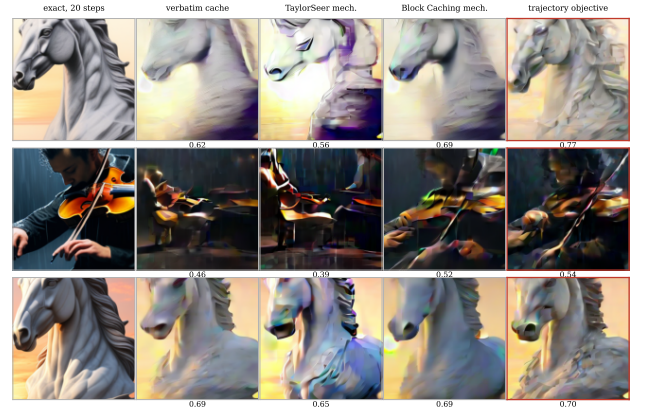


Figure 3. 250^2 crops at native resolution, same seed and prompt across a row, SSIM against the exact output below each panel. Crops rather than downsampled images, because thumbnails hide the texture differences these methods differ in.

is largest where residual training degrades worst: +0.0370 at $K=7$ against +0.0147 at $K=1$. This is consistent with the mechanism behind the degradation. A correction that reads the state makes error propagate across sites multiplicatively rather than additively, $\delta_{k+1} = (I + \sigma J_k)\delta_k + \varepsilon_k$, where δ_k is the deviation of the hidden state from the exact path on entering site k , ε_k is the residual error committed there, and $J_k = \partial \widehat{\Delta R}_k / \partial h$ comes from (4). Only Trajector observes the result of that compounding; residual training sees pointwise error. The degradation is weakened, not removed: the residual objective’s gain falls 3–4 $\times$ from $K=1$ to high K , while Trajector’s falls 1.2–1.8 $\times$.

Fig. 2 (right) also shows why σ must be re-selected per corrector. The optimum moves from 0.60 at $K=1$ to 0.18 at high K , and the peak narrows sharply: $K=4$ stays above +0.039 across $\sigma \in [0.18, 0.25]$ while $K=14$ and $K=28$ are already negative at $\sigma=0.25$. Both corrector generations trace the same family of curves, so this is a property of granularity rather than of the objective.

Fig. 3 shows the same three prompts. Failure modes differ in kind: verbatim caching smears texture into streaks; TaylorSeer’s artefact is chromatic fringing (worst LPIPS in Table 1); Block Caching flattens tone. Trajector recovers the background gradient most closely.

5.3. Scope and Limitations

On this backbone ImageReward saturates by 10 solver steps while pixel fidelity keeps improving, so a comparison at equal compute here mostly measures solver convergence. Holding the step count fixed isolates caching. Results are on 512^2 PixArt- Σ with templated prompts ($n=72$).

Schedules. All results use uniform anchors. Nothing in the method assumes them: C_θ is conditioned on τ rather than on i , and (6) is defined for any anchor set $\mathcal{A}$, so Trajector can be trained on the schedule it will be deployed with. Adaptive policies such as TeaCache or AdaCache couple the two. Their refresh decision reads the corrected state, so each corrector would choose its own schedule and compute budget. In a pilot with earlier $K=1$ correctors trained on residuals, we therefore froze a schedule in the style of TeaCache (refresh when accumulated change crosses a threshold, without its fitted rescaling), chosen on a rollout with no correction (e.g. anchors $\{0, 11, 15, 18\}$, so τ reaches 11 against 4 in training). Correction still helped (+0.008 and +0.021 SSIM for two training losses, $n=48$) but less than under uniform anchors (+0.036 and +0.047). We have not trained or tested Trajector on non-uniform schedules. Gains are after 400 optimiser steps without hyperparameter search, from a corrector trained on residuals, so they are what the objective change buys, not an upper bound. High- K correctors are deployed in fp32 to avoid fp16 overflow.

6. Conclusion

The natural objective for a learned cache corrector is to regress the residual it should have produced. That objective is misaligned with what caching damages: cache error integrates along the sampling trajectory, while residual error is pointwise. Fine-tuning the same corrector against trajectory deviation, at a cost of 20 minutes and no architectural change, improves image quality at every injection granularity we tested. At a fixed step count, learned correction is the only approach that improves on verbatim caching under a preference model, and Trajector leads on SSIM and PSNR.

Reproducibility

Code, configurations, logs and the per-case metrics behind every number are at <https://github.com/xiaruize0911/Diff-cache-Comp>. Every table cell can be recomputed from the archived `results.json`.

References

- [1] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *ICCV*, 2023.
- [2] X. Ma, G. Fang, and X. Wang, “Deepcache: Accelerating diffusion models for free,” in *CVPR*, 2024.
- [3] P. Selvaraju, T. Ding, T. Chen *et al.*, “Fora: Fast-forward caching in diffusion transformer acceleration,” *arXiv:2407.01425*, 2024.
- [4] P. Chen, M. Shen, P. Ye *et al.*, “ δ -dit: A training-free acceleration method tailored for diffusion transformers,” *arXiv:2406.01125*, 2024.
- [5] F. Liu, S. Zhang, X. Wang *et al.*, “Timestep embedding tells: It’s time to cache for video diffusion model,” *arXiv:2411.19108*, 2024.
- [6] J. Liu, C. Zou, Y. Lyu *et al.*, “From reusing to forecasting: Accelerating diffusion models with TaylorSeers,” in *ICCV*, 2025, *arXiv:2503.06923*.
- [7] F. Wimbauer, B. Wu, E. Schoenfeld *et al.*, “Cache me if you can: Accelerating diffusion models through block caching,” in *CVPR*, 2024.
- [8] X. Ma, G. Fang, M. B. Mi *et al.*, “Learning-to-cache: Accelerating diffusion transformer via layer caching,” in *NeurIPS*, 2024.
- [9] R. Rombach, A. Blattmann, D. Lorenz *et al.*, “High-resolution image synthesis with latent diffusion models,” in *CVPR*, 2022.
- [10] J. Chen, C. Ge, E. Xie *et al.*, “Pixart- σ : Weak-to-strong training of diffusion transformer for 4k text-to-image generation,” in *ECCV*, 2024.
- [11] P. Esser, S. Kulal, A. Blattmann *et al.*, “Scaling rectified flow transformers for high-resolution image synthesis,” in *ICML*, 2024.
- [12] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu *et al.*, “Flow matching for generative modeling,” in *ICLR*, 2023.
- [13] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *ICLR*, 2021.
- [14] C. Lu, Y. Zhou, F. Bao *et al.*, “Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps,” *NeurIPS*, 2022.
- [15] —, “DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models,” *arXiv:2211.01095*, 2022.
- [16] W. Zhao, L. Bai, Y. Rao *et al.*, “UniPC: A unified predictor-corrector framework for fast sampling of diffusion models,” in *NeurIPS*, 2023.
- [17] T. Salimans and J. Ho, “Progressive distillation for fast sampling of diffusion models,” *ICLR*, 2022.
- [18] Y. Song, P. Dhariwal, M. Chen *et al.*, “Consistency models,” in *ICML*, 2023.
- [19] S. Luo, Y. Tan, L. Huang *et al.*, “Latent consistency models: Synthesizing high-resolution images with few-step inference,” *arXiv:2310.04378*, 2023.
- [20] C. Zou, X. Liu, T. Liu *et al.*, “Accelerating diffusion transformers with token-wise feature caching,” *arXiv:2410.05317*, 2024.
- [21] Z. Lv, C. Si, J. Song *et al.*, “FasterCache: Training-free video diffusion model acceleration with high quality,” *arXiv:2410.19355*, 2024.
- [22] K. Kahatapitiya, H. Liu, S. He *et al.*, “Adaptive caching for faster video generation with diffusion transformers,” *arXiv:2411.02397*, 2024.
- [23] X. Shen, Z. Song, Y. Zhou *et al.*, “LazyDiT: Lazy learning for the acceleration of diffusion transformers,” in *AAAI*, 2025.
- [24] Y. Huang, Z. Wang, R. Gong *et al.*, “HarmoniCa: Harmonizing training and inference for better feature caching in diffusion transformer acceleration,” in *ICML*, 2025, *arXiv:2410.01723*.
- [25] J. Choi, J. Lee, C. Shin *et al.*, “Perception prioritized training of diffusion models,” in *CVPR*, 2022.
- [26] T. Hang, S. Gu, C. Li *et al.*, “Efficient diffusion training via min-snr weighting strategy,” in *ICCV*, 2023.
- [27] S. Ross, G. J. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *AISTATS*, 2011.
- [28] S. Bengio, O. Vinyals, N. Jaitly *et al.*, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *NeurIPS*, 2015.
- [29] Z. Wang, A. C. Bovik, H. R. Sheikh *et al.*, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, 2004.
- [30] R. Zhang, P. Isola, A. A. Efros *et al.*, “The unreasonable effectiveness of deep features as a perceptual metric,” in *CVPR*, 2018.
- [31] J. Xu, X. Liu, Y. Wu *et al.*, “ImageReward: Learning and evaluating human preferences for text-to-image generation,” in *NeurIPS*, 2023.